

TUTORIAL TÉCNICO

Configuración de Docker y CI/CD en GitHub

Proyecto: BDP PYME — Plataforma Virtual de Crédito

Repositorio:

Sistema-Crediticio-BDP.2.0

*Universidad Adventista de Bolivia
Ingeniería de Sistemas*

1. Introducción

Docker y los flujos de integración y despliegue continuos (CI/CD) son tecnologías que se han vuelto fundamentales en el desarrollo de software moderno. Su propósito es automatizar las etapas de construcción, prueba y despliegue de una aplicación, garantizando que el sistema funcione de manera idéntica en cualquier entorno (desarrollo, pruebas o producción), sin importar el sistema operativo del equipo donde se ejecute.

Docker resuelve el clásico problema de “en mi máquina funciona” al empaquetar la aplicación junto con todas sus dependencias dentro de un contenedor aislado. Por su parte, CI/CD a través de GitHub Actions permite que cada cambio enviado al repositorio sea validado automáticamente: se construye la imagen, se ejecutan verificaciones y se confirma que el sistema sigue operando correctamente antes de fusionarse con la rama principal.

En el presente tutorial se documenta paso a paso la implementación de estas tecnologías sobre el proyecto Sistema-Crediticio-BDP.2.0, abarcando los siguientes componentes:

- **Dockerfile** — definición de la imagen base y dependencias del sistema.
- **Docker Compose** — orquestación de servicios mediante contenedores.
- **GitHub Actions** — automatización de tareas en el repositorio.
- **Pipeline CI/CD** — flujo automatizado de construcción y validación.
- **Construcción automática de imágenes Docker** ante cada push al repositorio.

2. Objetivo

Implementar la contenedorización con Docker y la automatización del flujo de integración y despliegue continuo (CI/CD) dentro del repositorio de GitHub del proyecto Sistema-Crediticio-BDP.2.0, de modo que cada cambio enviado a la rama principal active de forma automática la construcción de la imagen Docker y la verificación del sistema.

3. Herramientas utilizadas

Las siguientes herramientas componen el entorno técnico del tutorial. Cada una cumple un rol específico dentro del flujo de trabajo:

Herramienta	Versión / Tipo	Función dentro del proyecto
Docker	Engine + Compose	Plataforma de contenedores para empaquetar la aplicación y sus dependencias.

Herramienta	Versión / Tipo	Función dentro del proyecto
GitHub	Repositorio remoto	Aloja el código fuente del proyecto y centraliza la colaboración del equipo.
GitHub Actions	CI/CD nativo	Ejecuta automáticamente el pipeline cada vez que ocurre un push a la rama main.
Visual Studio Code	Editor	Entorno de desarrollo donde se editan los archivos Dockerfile y de configuración.
Git	Control de versiones	Permite registrar y enviar cambios desde local hacia el repositorio remoto.

4. Apertura del proyecto en Visual Studio Code

Antes de iniciar cualquier configuración, el proyecto debe abrirse dentro de Visual Studio Code. Este editor proporciona acceso a la estructura de carpetas, terminal integrada y resaltado de sintaxis para los archivos de configuración que se crearán más adelante.

Pasos a seguir

- Abrir Visual Studio Code desde el menú de inicio del sistema.
- Seleccionar la opción **File** → **Open Folder** desde la barra superior.
- Navegar hasta la carpeta del proyecto `Sistema-Crediticio-BDP.2.0` y abrirla.

Una vez abierta la carpeta, el panel lateral izquierdo (Explorer) mostrará la estructura inicial del repositorio. En este caso, el proyecto contiene la carpeta docs con sus subdirectorios de avances, diagramas, historias de usuario y documentos finales, además del archivo `.gitkeep`.

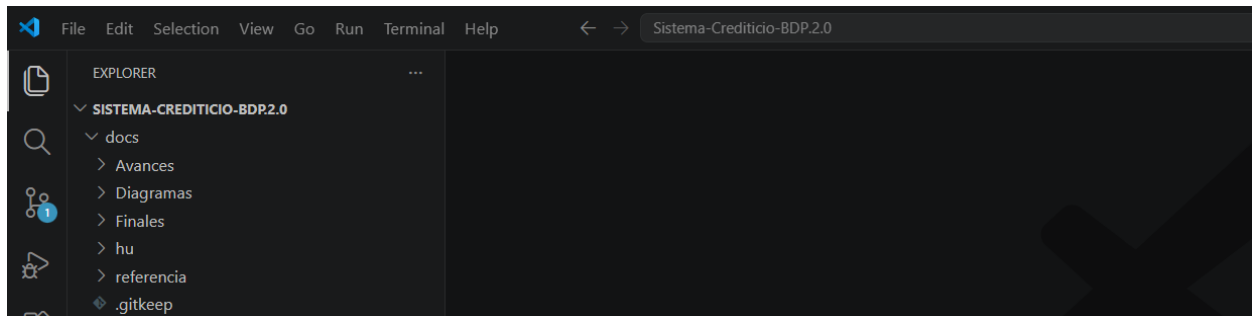


Figura 1. Estructura inicial del proyecto Sistema-Crediticio-BDP.2.0 en VS Code.

5. Creación del Dockerfile

El Dockerfile es un archivo de texto plano que contiene las instrucciones necesarias para construir una imagen de Docker. En él se declara qué sistema base utilizará el contenedor, qué dependencias instalar, qué archivos copiar y qué comando ejecutar al iniciar la aplicación. Sin este archivo, Docker no sabría cómo empaquetar el proyecto.

Pasos para crearlo

- En el panel Explorer de VS Code, hacer clic derecho sobre el nombre del proyecto.
- Seleccionar la opción **New File** (Nuevo archivo).
- Asignar el nombre exacto **Dockerfile** (sin extensión, con la D en mayúscula).

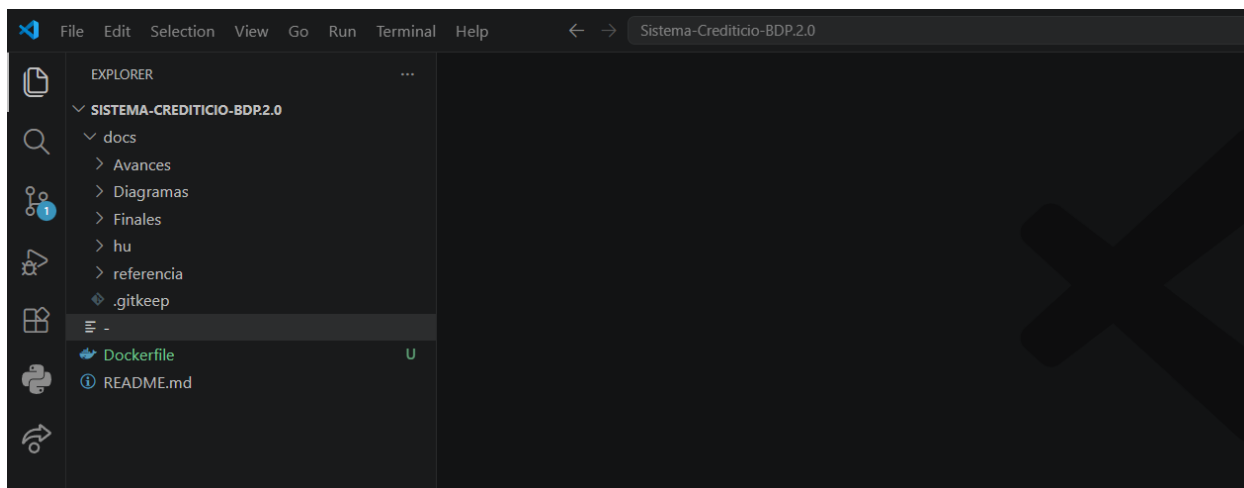


Figura 2. Archivo Dockerfile creado en la raíz del proyecto.

6. Configuración del Dockerfile

Una vez creado el archivo, se escribió el siguiente contenido. Cada línea define una etapa en la construcción de la imagen que Docker utilizará para ejecutar la aplicación:

```
FROM python:3.11

WORKDIR /app

COPY requirements.txt .

RUN pip install -r requirements.txt

COPY . .

EXPOSE 5000

CMD ["python", "app.py"]
```

Explicación detallada de cada instrucción

Instrucción	Significado y propósito
FROM python:3.11	Define la imagen base que se descargará de Docker Hub. En este caso se utiliza la versión oficial de Python 3.11, que incluye el intérprete y las librerías estándar.
WORKDIR /app	Establece el directorio de trabajo dentro del contenedor. Todas las instrucciones posteriores se ejecutarán desde /app.

Instrucción	Significado y propósito
COPY requirements.txt .	Copia únicamente el archivo de dependencias al contenedor. Se copia primero para aprovechar el sistema de caché de capas de Docker.
RUN pip install -r requirements.txt	Ejecuta pip dentro del contenedor para instalar todas las librerías listadas en requirements.txt.
COPY . .	Copia el resto del proyecto al directorio de trabajo del contenedor.
EXPOSE 5000	Declara que el contenedor escuchará en el puerto 5000. No abre el puerto al exterior por sí solo, solo lo documenta.
CMD ["python", "app.py"]	Indica el comando que se ejecutará automáticamente cuando el contenedor inicie: lanzar la aplicación con Python.

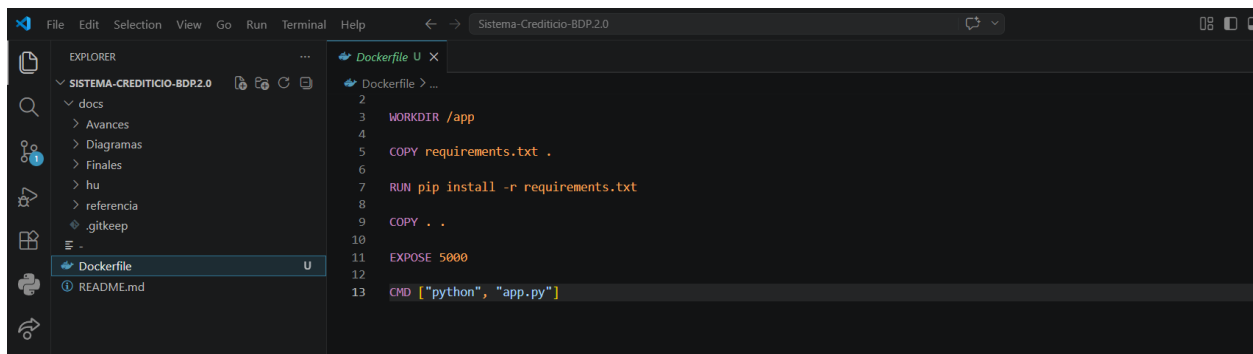


Figura 3. Contenido completo del archivo Dockerfile en Visual Studio Code.

7. Creación del archivo docker-compose.yml

Docker Compose es una herramienta complementaria que permite definir y ejecutar aplicaciones multi-contenedor mediante un único archivo YAML. Aunque en este proyecto el sistema utiliza un solo servicio, Docker Compose facilita su ejecución con un comando único y deja preparada la estructura para incorporar otros servicios en el futuro (por ejemplo, una base de datos o un servidor web).

Pasos

- Crear un nuevo archivo en la raíz del proyecto llamado **docker-compose.yml**.
- Escribir el siguiente contenido respetando la indentación con espacios (no tabulaciones).

```
version: '3'

services:
  app:
    build: .
    ports:
```

```
- "5000:5000"
```

Explicación

- `version: '3'` — define la versión de la sintaxis de Compose utilizada.
- `services:` — bloque que agrupa todos los contenedores que componen la aplicación.
- `app:` — nombre lógico del servicio principal.
- `build: .` — indica que la imagen se construirá a partir del Dockerfile ubicado en el directorio actual.
- `ports: - "5000:5000"` — mapea el puerto 5000 del contenedor al puerto 5000 del equipo anfitrión.

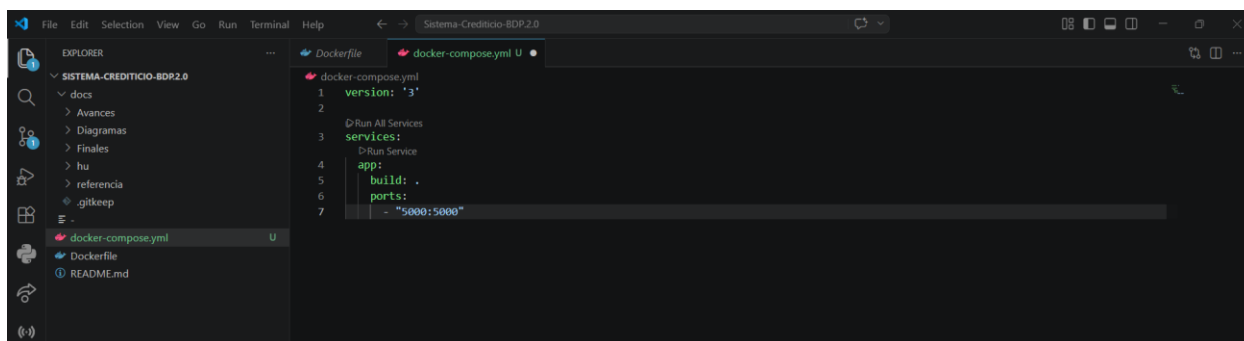


Figura 4. Archivo docker-compose.yml creado y configurado en VS Code.

8. Configuración de GitHub Actions

GitHub Actions es el servicio integrado de GitHub para automatizar tareas dentro de un repositorio. Permite definir flujos de trabajo (workflows) escritos en YAML que se ejecutan automáticamente cuando ocurre un evento, como un push, un pull request o la creación de un release. En este proyecto se utilizará para construir la imagen Docker cada vez que se envíen cambios a la rama principal.

Para que GitHub Actions reconozca el flujo, los archivos deben colocarse dentro de una ruta específica: `.github/workflows`.

8.1 Crear la estructura de carpetas

- En la raíz del proyecto, crear una carpeta llamada `.github` (con el punto al inicio).
- Dentro de ella, crear una segunda carpeta llamada `workflows`.

La estructura final debe quedar así:

```
Sistema-Crediticio-BDP.2.0/  
└─ .github/  
    └─ workflows/
```

9. Creación del pipeline CI/CD

Dentro de la carpeta `.github/workflows` se crea el archivo `ci-cd.yml`, que define las etapas del flujo de integración continua. Cada vez que ocurra un push a la rama `main`, GitHub Actions ejecutará este flujo de forma automática en un servidor de Ubuntu.

```
name: CI/CD Pipeline  
  
on:  
  push:  
    branches:  
      - main    # Sincronizado con la rama local  
  
jobs:  
  build:  
    runs-on: ubuntu-latest  
  
    steps:  
      - name: Clonar repositorio  
        uses: actions/checkout@v3  
  
      - name: Configurar Docker
```



```
uses: docker/setup-buildx-action@v2

- name: Construir imagen Docker
  run: docker build -t sistema-bdp .

- name: Verificar imágenes
  run: docker images
```

Explicación del pipeline paso a paso

Sección	Función
name	Asigna un nombre identificable al workflow. Aparecerá en la pestaña de GitHub Actions.
on: push	Define el evento disparador: el flujo se activa al hacer push a la rama indicada.
branches: main	Especifica que el pipeline solo se ejecuta cuando los cambios se envían a la rama main.
runs-on: ubuntu-latest	Indica que el trabajo se ejecuta en una máquina virtual con Ubuntu en su versión más reciente.
actions/checkout@v3	Acción oficial de GitHub que descarga el código del repositorio dentro del runner.
docker/setup-buildx-action@v2	Prepara el entorno con Docker Buildx, necesario para construir imágenes.
docker build -t sistema-bdp .	Construye la imagen del proyecto y le asigna la etiqueta sistema-bdp.
docker images	Lista las imágenes existentes para confirmar que la construcción fue exitosa.

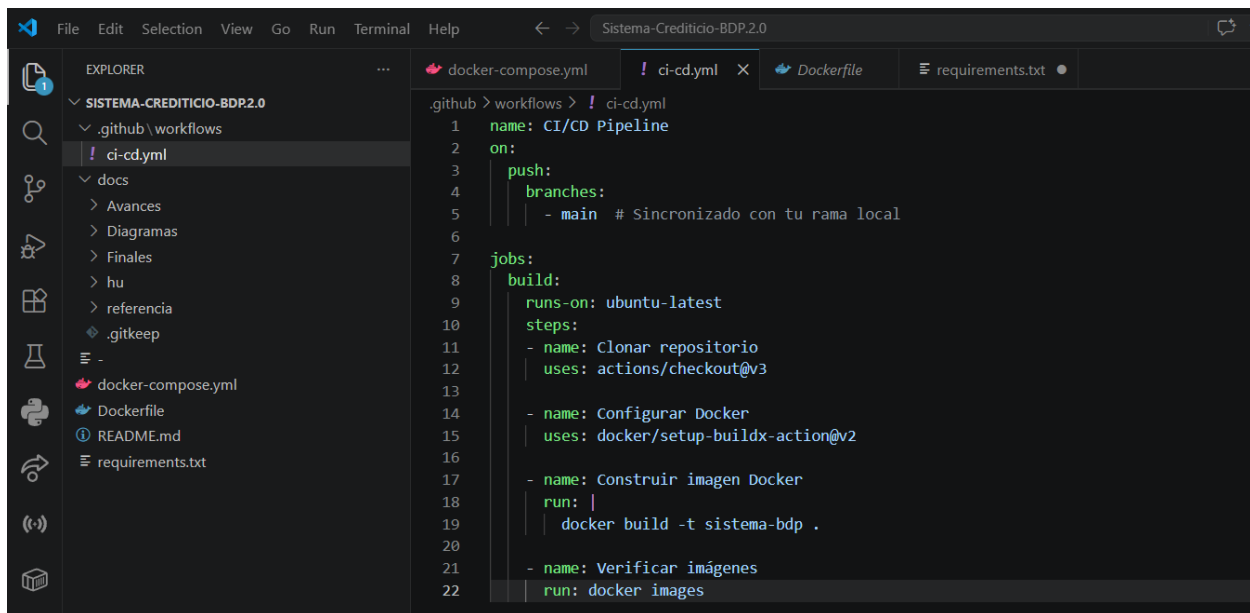


Figura 5. Contenido del archivo ci-cd.yml dentro de .github/workflows.

10. Guardado de cambios en GitHub

Una vez creados el Dockerfile, el docker-compose.yml y el pipeline ci-cd.yml, todos los archivos deben enviarse al repositorio remoto en GitHub para que el flujo de trabajo se active. Esto se realiza mediante los comandos estándar de Git.

Comandos ejecutados en la terminal de VS Code

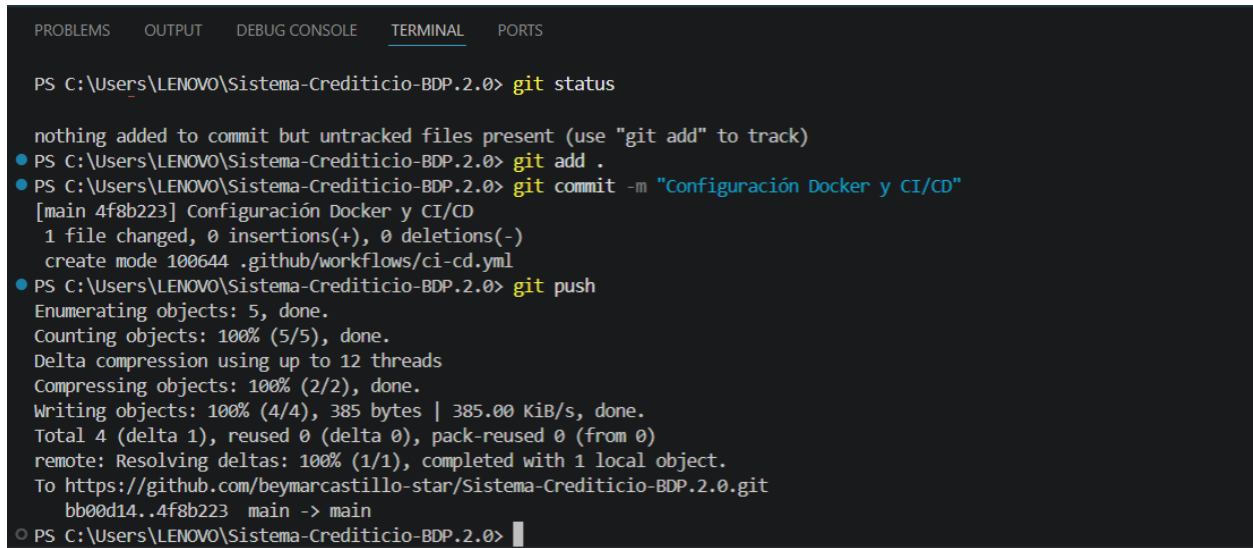
```

git status
git add .
git commit -m "Configuración Docker y CI/CD"
git push

```

Significado de cada comando

- **git status** — muestra el estado actual del repositorio y los archivos modificados o sin seguimiento.
- **git add .** — añade todos los cambios al área de preparación (staging).
- **git commit -m "..."** — registra los cambios en el historial local con un mensaje descriptivo.
- **git push** — envía los commits locales hacia el repositorio remoto en GitHub.



The image shows a terminal window from Visual Studio Code with the 'TERMINAL' tab selected. The terminal displays the output of a series of Git commands executed in a PowerShell prompt. The commands and their outputs are as follows:

```
PS C:\Users\LENOVO\Sistema-Crediticio-BDP.2.0> git status

nothing added to commit but untracked files present (use "git add" to track)
● PS C:\Users\LENOVO\Sistema-Crediticio-BDP.2.0> git add .
● PS C:\Users\LENOVO\Sistema-Crediticio-BDP.2.0> git commit -m "Configuración Docker y CI/CD"
[main 4f8b223] Configuración Docker y CI/CD
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 .github/workflows/ci-cd.yml
● PS C:\Users\LENOVO\Sistema-Crediticio-BDP.2.0> git push
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (4/4), 385 bytes | 385.00 KiB/s, done.
Total 4 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/beymarcastillo-star/Sistema-Crediticio-BDP.2.0.git
 bb00d14..4f8b223  main -> main
○ PS C:\Users\LENOVO\Sistema-Crediticio-BDP.2.0> 
```

Figura 6. Terminal de VS Code ejecutando los comandos `git add`, `git commit` y `git push`.

11. Verificación del pipeline en GitHub

Inmediatamente después del push, GitHub Actions detectó los cambios en la rama main y ejecutó el pipeline definido en ci-cd.yml. Para confirmar que la ejecución fue exitosa se debe acceder al repositorio en GitHub y consultar la pestaña correspondiente.

Pasos para verificar

- Ingresar al repositorio en la plataforma de GitHub.
- Seleccionar la pestaña **Comportamiento** (Actions, en su versión en español).
- Localizar la ejecución más reciente del flujo *Pipeline final con requisitos y rama principal*.
- Verificar que aparezca con el indicador verde ✓, que confirma una ejecución sin errores.

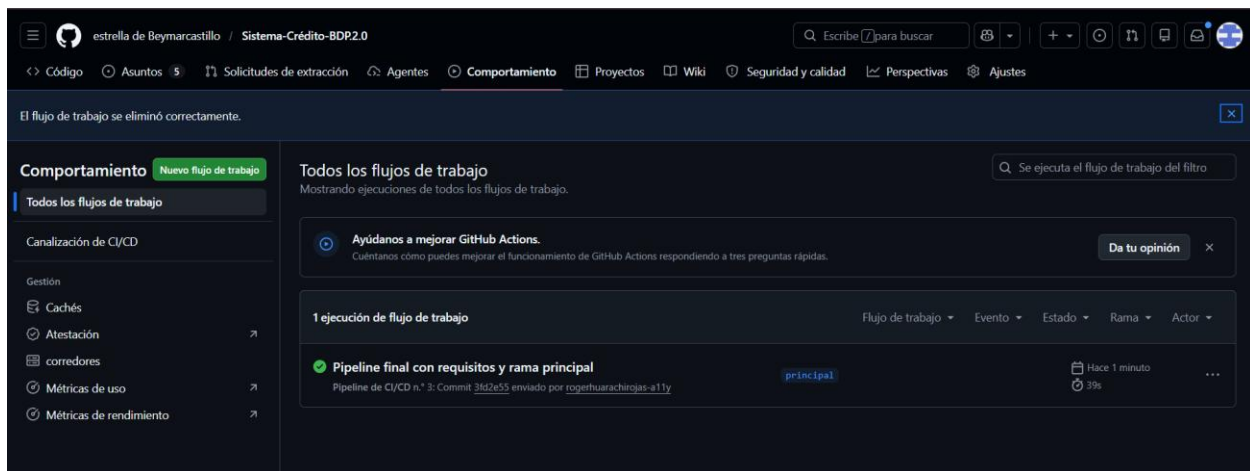


Figura 7. Ejecución exitosa del pipeline CI/CD en la pestaña Comportamiento de GitHub.

12. Resultados obtenidos

Al finalizar la implementación se alcanzaron los siguientes resultados, todos verificables tanto en el entorno local como en el repositorio remoto:

- **Contenerización completa de la aplicación.** El sistema BDP PYME ahora puede ejecutarse dentro de un contenedor Docker sin depender de instalaciones manuales de Python o dependencias en el equipo anfitrión.
- **Automatización mediante GitHub Actions.** Cada push a la rama main dispara el pipeline sin intervención manual.
- **Integración continua funcional.** La construcción y verificación de la imagen ocurre de forma automática y en un entorno limpio.
- **Reducción de errores manuales.** Al estandarizar el proceso, se eliminan inconsistencias entre los equipos del grupo de trabajo.

- **Despliegues más rápidos y consistentes.** Cada nueva versión queda validada y lista para ser desplegada.

13. Conclusiones

La implementación de Docker y de un pipeline CI/CD con GitHub Actions sobre el proyecto Sistema-Crediticio-BDP.2.0 permitió automatizar tareas críticas que antes se realizaban de forma manual, como la construcción de la aplicación y la verificación de su correcto funcionamiento ante cada cambio.

Docker garantizó la portabilidad y consistencia del entorno de ejecución, eliminando los problemas asociados a configuraciones distintas entre los equipos del grupo. Por su parte, GitHub Actions integró este proceso directamente en el flujo de trabajo del repositorio, brindando retroalimentación inmediata sobre el estado del sistema.

En conjunto, ambas tecnologías mejoraron de manera significativa la eficiencia del equipo, la escalabilidad de la solución y la confiabilidad del sistema financiero BDP PYME, sentando una base sólida para futuras integraciones como pruebas automatizadas, análisis estático de código o despliegues a entornos de producción.